

Gestion de code avec Git (FIEC30EM)

- Le projet -

frank.buloup@univ-amu.fr

Aix Marseille Université
Faculté des Sciences du Sport
Master 2 IEAP - Facteurs Humains



Gestion de code avec Git - Le projet

- 1 Présentation - Consignes générales
- 2 Les étapes à suivre
- 3 Pour aller plus loin sur les différents postes

1 Présentation - Consignes générales

Rien à disposition pour faire des mesures objectives ?

- Microcontrôleur (Arduino, BeagleBone Black, M5Stack etc.)
- Capteurs, Fer à souder, fils etc.
- Revendeurs :
 - Gotronic
 - Lextronic
 - Radiospare
 - Farnell
 - SparkFun Electronics
 - Seeed Studio
 - Digi-Key
 - Mouser
 - etc.

DEVIS

COORDONNÉES DE FACTURATION

Aix Marseille Université
Buloup Frank
Faculté des Sciences du Sport
163 avenue de Lumina Case 910
13288 Marseille Cedex 9
France métropolitaine

frank.buloup@univ-amu.fr

Panier N°	1540892
-----------	---------

Référence	Désignation	PU HT	PU TTC	Qté	Total HT	Total TTC
38375	Arduino Uno R4 WiFi ABX00087	25,00 €	30,00 €	12	300,00 €	360,00 €
25556	Capteur de force 5 kg CZL635-5	7,92 €	9,50 €	2	15,83 €	19,00 €
34732	Amplificateur pour capteur de force SEN-13879	10,75 €	12,90 €	2	21,50 €	25,80 €
32772	Capteur de force FSR402S	8,00 €	9,60 €	2	16,00 €	19,20 €
24504	Transducteur à ultrasons SRF05	13,83 €	16,60 €	2	27,67 €	33,20 €

Figure: Exemple de devis - 10/09/2024

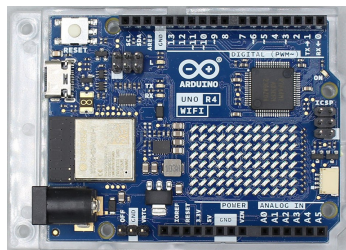


Figure: Arduino Uno R4 Wifi

Projet global sur la base d'un Arduino Uno R4 Wifi

- Création d'un gestionnaire d'essais en Python
- Acquisition, visualisation et traitement des signaux
- Répartir les tâches dans l'équipe
- Gérer les codes avec Git sur GitHub
- Ne pas oublier de calibrer le capteur lorsque c'est nécessaire

Gestionnaire d'essais

Nombre d'essais :

Préfixe des noms des fichiers de données :

Répertoire de destination :



Essai courant : Etat

A effectuer
En cours

Figure: Exemple d'IHM du gestionnaire d'essais

IHM : Interface Homme Machine (Graphical User Interface)

Conseils & consignes !

- Utilisez Arduino IDE pour programmer : port USB C
- Utilisez ce même port pour communiquer avec l'arduino
- Utilisez Python pour la partie PC (IHM avec Tkinter)
- Utilisez la librairie pySerial de Python
- Utilisez l'IDE de votre choix pour Python
- Assurez-vous que le moniteur série (Arduino IDE) est fermé avant de lancer votre programme Python
- Suivez les étapes ci-après : chaque étape est un commit
- Le code générique pour un essai **doit** fonctionner **SANS** l'IHM
- Le développement de l'IHM viendra dans un second temps
- **Chaque étape fera l'objet d'un commit sur GitHub**

2 Les étapes à suivre

GIT Step 1 - Synchronisation 1

- Attendre la réception d'un caractère spécial pour démarrer la boucle loop() de l'Arduino
- Ce step comportera également la configuration de la liaison série entre le PC et l'Arduino à 1Mbits/s, 8 bits de données, pas de parité et 1 bit de stop
- Vous pourriez utiliser la LED interne (LED_BUILTIN) pour vérifier que vous êtes bien entré dans la boucle

Universal Asynchronous Receiver/Transmitter

- Point sur les UART/USART
- SPI, I2C (TWI), 1-WIRE etc.

Pour plus d'infos :

[Communication série et UART/USART](#)

Étape 1

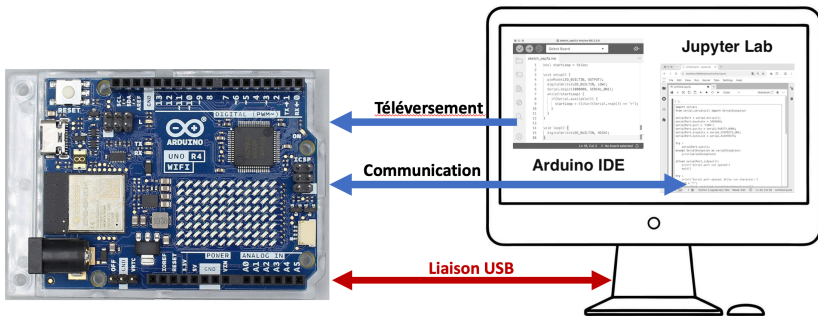


Figure: Résumé étape 1

GIT Step 2 - Synchronisation 2

- Dans la boucle loop(), redémarrer l'Arduino dès réception d'un caractère spécial (Software reset)
- Le redémarrage aura lieu après 10 secondes environ et sera initié côté PC

GIT Step 3 - Comptage temporel

- Mettre en place un système de comptage et de réinitialisation du temps en μs dans l'arduino
- La réinitialisation aura lieu à chaque redémarrage de l'Arduino
- Vous pourrez utiliser les fonctions `Serial.println()` et `delay()` pour vérifier votre code

GIT Step 4 - Transmission du comptage temporel

- Transmettre ce comptage de temps de l'Arduino au PC
- Vous utiliserez les fonctions `sprintf()` et `Serial.println()`
- Supprimez les sorties consoles python qui prennent du temps
- Supprimez les `delay()` côté Arduino si utilisés
- Utilisez la commande flush du port série (Arduino et Python)
- Vous accumulerez les valeurs temporelles dans un tableau `times[]` en secondes

GIT Step 5 - Transmission valeurs du capteur

FSR (pour les deux postes) ou Potentiomètre

- Lire la valeur brute de la FSR ou du potentiomètre
- Transmettre la valeur brute (SFR) ou calibrée (Potar) au PC
- Vous utiliserez les mêmes fonctions `sprintf()` et `Serial.println()`

GIT Step 5 - Transmission valeurs du capteur

Jauge de contrainte (pour les deux postes)

- Calibrer la jauge avec les petites masses
- Lire la valeur brute de la jauge
- Transmettre la valeur calibrée au PC
- Vous utiliserez les mêmes fonctions `sprintf()` et `Serial.println()`

GIT Step 5 - Transmission valeurs du capteur

Capteur ultrason (pour les deux postes)

- Utiliser la librairie de [Rob Tillaard](#)
- Transmettre la valeur de distance en cm
- Vous utiliserez les mêmes fonctions `sprintf()` et `Serial.println()`

GIT Step 5 - Transmission valeurs du capteur

Codeur Optique

- Utiliser le concept d'interruption pour réagir aux changements sur les entrées A et B du codeur
- Dans les fonctions d'interruption, réinitialiser la valeur de l'angle si le signal Z du capteur est actif
- Dans les fonctions d'interruption, utiliser les valeurs courantes et précédentes de A et B pour mettre à jour la valeur angulaire du capteur
- Transmettre cette valeur au pc (dans la boucle loop())
- Vous utiliserez les mêmes fonctions `printf()` et `Serial.println()`

Réagir à l'état d'une entrée : Polling (sondage) & Interruption

Pour plus d'infos : [Renesas - Interrupt](#)

GIT Step 6 - Accumulation, affichage et enregistrement des valeurs

- Vous avez accumulé durant les dix secondes ces valeurs (signaux et temps) dans un tableau
- Afficher le signal à la fin de ces dix secondes (graphe temporel)
- Enregistrer les valeurs dans un fichier

GIT Step 7 - Réaliser plusieurs essais

- Ajouter une boucle pour réaliser plusieurs essais (e.g. 10 essais)
- Prendre en compte le numéro d'essai dans le nom des fichiers de données

GIT Step 8 - IHM

- Ajouter une IHM permettant de paramétrer l'acquisition

GIT Steps suivants si possible

En vous appuyant sur les propositions ci-après, continuer en choisissant de vous répartir le travail de développement et de gestion de code en autonomie.

- ### 3 Pour aller plus loin sur les différents postes
- Jauge de contrainte - 2 postes
 - Résistance à détection de force - Temps de réponse
 - Résistance à détection de force - Comptage
 - Capteur à ultrason - 2 postes
 - Potentiomètre - Pendule simple*
 - Codeur optique*

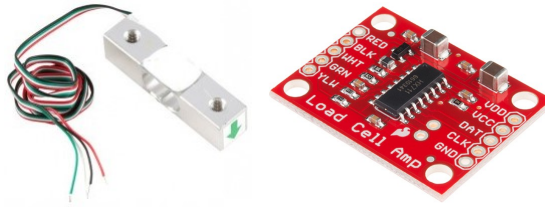


Figure: La jauge (CZL635-5) et son ampli (SEN-13879)

TODO en plus ! (si possible ?)

- Faire défiler la valeur de la masse sur la matrice LEDs de l'Arduino
- Afficher la valeur de la masse sur une page Web en utilisant le Wifi de l'arduino et un point d'accès mobile sur votre ordi ou téléphone



Figure: La résistance à détection de force (FSR402S)

TODO en plus ! (si possible ?)

- Générer un top aléatoires avec la DEL de l'Arduino dans un intervalle de temps de 1 à 9 secondes. Le top durera 1 seconde.
- Taper sur la FSR dès que le top est perçu
- Enregistrer ces deux signaux (DEL et FSR)
- Caractériser leur écart temporel



Figure: La résistance à détection de force (FSR402S)

TODO en plus ! (si possible ?)

- Taper sur la FSR régulièrement
- Compter le nombre de tapes



Figure: Le capteur à ultrason (SRF05)

TODO en plus ! (si possible ?)

- Faire défiler la valeur de la distance sur la matrice LEDs de l'Arduino
- Faire clignoter la LED de l'Arduino en liant la cadence de clignotement à la distance donnée par le capteur à ultrason



Figure: Le potentiomètre (AHM5)

TODO en plus ! (si possible ?)

- Enregistrer pour plusieurs positions de la masse
- Estimer la longueur du bras d'oscillation pour chaque position
- Identifier un système qui simule le mouvement



Figure: Le codeur optique

TODO en plus ! (si possible ?) Retrogaming Pong ;-)

- Positionner une raquette du jeu en fonction de la valeur envoyée par le codeur